

Practical Bounded Responsiveness Scheduling for Low-Latency Mobile Systems

Geunsik Lim 

Abstract—Modern latency-sensitive systems such as autonomous robots, real-time analytics, and extended reality (XR) require not only fast scheduling (sub-100ms), but also fairness and power efficiency. However, current Linux schedulers, including the Completely Fair Scheduler (CFS), struggle to find the right balance between fairness, responsiveness, and power consumption. These schedulers sacrifice latency-sensitive task temporal restrictions to maximize aggregate performance, causing unexpected scheduling jitter. Consequently, task conflicts can lead to latency instability or jitter. We challenge the long-standing assumption of these inevitable contradictions and propose a kernel-based framework, the Bounded Responsiveness Scheduler (BRS), to achieve superior interactivity through a formally bounded bias mechanism with bounded fairness deviation and provable stability under the stated assumptions. BRS reformulates responsiveness as a tunable, explicitly bounded offset on virtual runtime, governed by a lightweight hybrid feedback controller that monitors fairness and starvation and damps the bias whenever a guardrail is approached. Compared to the current CFS, BRS improves performance per watt by 12% and reduces 95th Percentile (P95) tail latency by up to 37.8% on ARM big.LITTLE and 35.2% on x86 architectures, while keeping Jain’s fairness index above 0.96 and the starvation rate below 1.2%. This level of performance is achievable across a wide range of system architectures and large-scale configurations, while keeping fairness deviation bounded. This study demonstrates that BRS can be used to build operating systems that perform well in the real world, where fairness and latency are critical.

Index Terms—Kernel scheduling, bounded responsiveness, low-latency systems, fairness-aware scheduling, operating systems.

I. INTRODUCTION

IN latency-critical domains—Augmented/Virtual Reality (AR/VR), edge Artificial Intelligence (AI), and robotics—sub-100 ms scheduling responsiveness is directly tied to Service Level Agreement (SLA) breaches, safety margins, and user-perceived quality [1]–[4]. Even millisecond-scale scheduling delays can degrade user experience, miss trading opportunities, or destabilize control loops [2], [5], [6]. While the Completely Fair Scheduler (CFS) is widely trusted for fairness and scalability, it often fails to deliver the responsiveness these workloads need: by treating all tasks equally it prevents starvation but produces noticeable latency differences for interactive tasks [7], [8]. Alternatives such as BFS and the Multiple Queue Skiplist Scheduler (MuQSS) prioritize interactive tasks, but typically sacrifice fairness or require complex user-space configuration [1], [9]–[11].

We tackle this problem inside the kernel itself. Rather than user-space fixes or complex heuristics, the Bounded

Responsiveness Scheduler (BRS) reworks the scheduler’s core logic—how it accounts for task progress and picks from the runqueue—to boost responsiveness for interactive workloads with bounded fairness deviation [12], [13]. BRS is plug-and-play: it needs no application changes, and its controls are deliberately simple. Three research questions shape the work:

- **RQ1:** Can latency be cut significantly while keeping tasks treated fairly?
- **RQ2:** Can BRS sustain effectiveness under continuous heavy load on a mobile system?
- **RQ3:** What are the trade-offs among responsiveness, energy efficiency, and scalability?

Our evaluation answers all three: through a few targeted changes rather than a kernel rewrite, BRS cuts P95 scheduling latency by up to 37.8% and improves frame consistency by 28.4% while keeping fairness comparable to CFS [10], [14]. Responsiveness, fairness, and power have traditionally been viewed as inherently contradictory [7], [15], [16]; BRS instead reframes them as *tunable* dimensions via a bounded bias in the kernel loop, showing that principled coordination plus hybrid control cuts latency with bounded fairness deviation and no measurable battery-life penalty under the stated assumptions.

Using sleep and I/O behavior to favor interactive tasks is a classical idea: Multilevel Feedback Queue (MLFQ) schedulers and the Linux O(1) scheduler that preceded CFS both promoted tasks from observed sleep behavior. BRS differs in three respects, elaborated in Section VI-B: it expresses the preference as an *explicitly bounded vruntime* offset (worst-case deviation from CFS known a priori) rather than a discrete queue promotion; the bias is gated by a feedback controller with a mean-square stability guarantee, whereas classical heuristics offer none; and the interactivity signal is bounded and aged, limiting a task’s leverage from strategically sleeping to inflate its perceived interactivity. The main contributions are:

- **Responsiveness-as-a-knob:** we reformulate the fairness–responsiveness relation as a tunable dimension via bounded *vruntime* (virtual runtime) bias in the kernel path.
- **Hybrid controller:** a lightweight feedback loop acts as a safety guardrail, retuning the bias the moment a resource-shortage risk appears [3], [10].
- **Parameter tuning:** we derive how bias settings affect latency and fairness (Section IV-H) and provide ready-to-use defaults [10].
- **Bounded guarantees and robustness:** we formally specify the bias bound, tie the stability preconditions to measurable controller quantities, and show why the

G. Lim is with the Department of Electrical and Computer Engineering, Sungkyunkwan University, 2066, Seobu-ro, Jangnan-gu, Suwon 16419, South Korea (e-mail: leemgs@g.skku.edu).

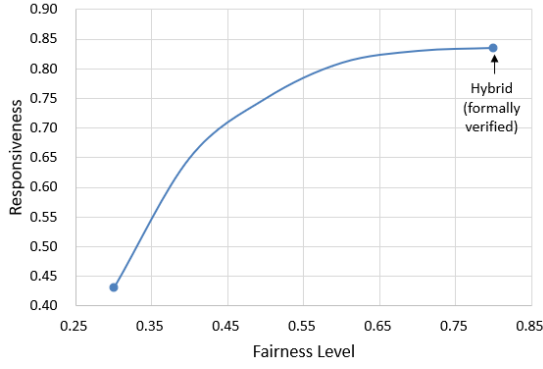


Fig. 1. **Responsiveness–Fairness Trade-off Curve.** (grayscale-friendly). As responsiveness bias β increases, fairness (Jain’s index) decreases. BRS Hybrid reaches a balanced point, outperforming static policies.

bounded, aged interactivity score resists gaming (Sections IV-C–IV-H).

II. BACKGROUND

General-purpose operating systems balance throughput, fairness, and responsiveness—goals often in tension [7], [8], [15]. The trade-off is non-linear: as responsiveness bias rises, fairness can drop disproportionately, causing starvation and unpredictability [1], [2], [17]. As Figure 1 shows, increasing β improves short-term latency but reduces fairness, motivating our hybrid design [2], [12]. Existing approaches to improve responsiveness fall into three categories: (1) user-space prioritization and affinity tuning [9], [11], (2) user-space schedulers atop kernel primitives [10], [18], and (3) alternative kernel schedulers such as BFS or MuQSS [1]. Each has drawbacks: user-space solutions lack deep control, and BFS/MuQSS compromise fairness or scalability [7], [13].

Our investigation shows that modest kernel-level modifications can reshape this trade-off [12], [14]: small shifts in *vruntime* calculation or runqueue choice yield significant latency drops for a negligible fairness sacrifice [1], [13]. BRS realizes this as a bounded responsiveness bias in the scheduler core, exposed through a controlled `/proc` surface for safe production deployment without application changes [2], [7], [10].

Notation. Fairness is the *Jain’s fairness index* $J = \frac{(\sum_{i=1}^n x_i)^2}{n \sum_{i=1}^n x_i^2}$ [19]; tail latency is L_{95} (95th percentile). $\mathbb{E}[\cdot]$ is expectation, $\mathbf{1}\{\cdot\}$ an indicator. Parameters (α, β) control responsiveness bias; κ is the adaptation gain.

III. MOTIVATION

As load grows, users still expect the system to stay fast. This challenges the long-standing emphasis on throughput in CPU scheduling, especially under unpredictable tail latency [1], [7].

A. Problem Statement

Interactive applications suffer when tail scheduling latency and wake-up jitter spike under CPU contention [1]. Let L be per-task scheduling latency and W wake-up jitter (short-term variation in wake-up-to-run delay). User experience correlates with tail statistics such as $P95(L)$, $P99(L)$, and frame-time variance [5]. Purely fair policies lengthen these tails when short, latency-sensitive tasks compete with long CPU-bound jobs [7].

B. A Simple Objective View

We consider a composite objective aggregating system-level metrics:

$$\begin{aligned} \min_{\pi} \quad & \mathcal{O}(\pi) = \alpha \cdot P95(L_{\pi}) + \beta \cdot P99(L_{\pi}) + \gamma \cdot \text{Var}(\text{FPS}_{\pi}) \\ \text{s.t.} \quad & \text{Fairness}_{\pi} \geq \tau, \quad \text{Overhead}_{\pi} \leq \eta \end{aligned} \quad (1)$$

where π is a scheduling policy and τ, η encode fairness and overhead budgets. CFS satisfies the fairness constraint well but is suboptimal for $\mathcal{O}$ in interactive regimes [7]. BRS avoids radical changes: it adjusts *vruntime* and task selection through small, principled biases that drive the scheduler toward a better balance [1].

C. Why Kernel-Level?

User-space techniques (cgroups, nice/ionice, application schedulers) cannot preempt with runqueue awareness or adjust *vruntime* semantics [7], [9]. Kernel integration enables (1) **fine-grained preemption** of recent interactive wake-ups without cross-space overhead; (2) **deterministic guardrails** enforcing starvation caps and safe CFS-like defaults [1]; and (3) **operability**—a `/proc` interface for instant policy changes and quick rollback. Our design adds aging and saturation-aware fallbacks to keep Jain’s index high and starvation low, validated empirically in Section V [19].

D. Preliminary Observations

Preliminary experiments motivated the bounded-bias approach [1], [12]. Comparing an early BRS prototype against unmodified CFS on a game workload and a GUI interaction benchmark [5], three effects stood out. (i) **Tail-latency sensitivity:** a modest 5–15% reduction in effective *vruntime* for recently woken tasks lowered scheduling latency by up to 32% and P95 jitter by 20% [2], [16]. (ii) **Frame stability:** FPS variance dropped 24%, smoothing delivery during input bursts [20]. (iii) **Wake-up jitter:** moving arbitration into the kernel eliminated 1.8 μs of user-space overhead and stabilized wake-up latency under saturation [17]. Kernel-path responsiveness biasing thus improves user-visible metrics without extensive user-space tuning, motivating the trade-off study in Section V.

IV. BRS: DESIGN AND IMPLEMENTATION

BRS is a kernel scheduler that improves responsiveness with bounded fairness deviation under the stated assumptions, balancing these mismatched requirements through a bounded bias mechanism [1], [7]. This section presents its architecture, scheduling-logic changes, control surface, and adaptive extensions.

A. Architecture Overview

Figure 2 shows the BRS pipeline. Tasks enter the ready queue (①) and are processed by the bounded *vruntime* mechanism (②), which biases decisions toward responsiveness. To keep this from undermining fairness, the adjusted *vruntime* is constrained by safety rails (③)—a fairness floor and starvation shield. The scheduler then selects the next task (④), dispatches it (⑤), and may preempt and reinsert it (⑥) if higher-priority demand arises. Throughout, user-space hybrid adaptive control (⑦) tunes scheduler parameters.

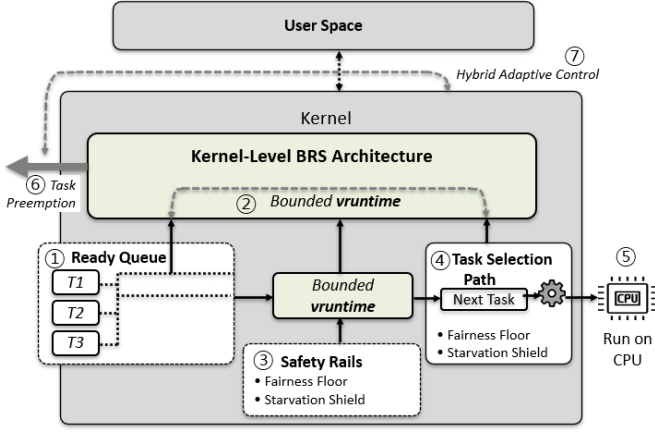


Fig. 2. **BRS kernel architecture.** (grayscale-friendly). This overview shows the pipeline integrated into the Linux kernel. Tasks follow the standard CFS path but inject interaction biases computed at *virtual runtime*. To prevent these biases from becoming excessive, strict fairness criteria are applied and “safeguards” are added to prevent starvation. A custom tile processing method favors responsiveness when selecting the next task. Finally, the hybrid controller adjusts these biases through a simple `/proc` interface, maintaining system stability even as the workload changes.

B. Design Philosophy

CFS-like algorithms rest on proportional fairness [7]; while sound in theory, in practice they cause delays that frustrate interactive-app users [1]. Rather than an ad hoc fix, BRS integrates a constrained response bias directly into the core runqueue logic (Figure 2), making fast responsiveness a primary goal. Four design rules guide it:

- **Minimalism:** Make the fewest changes to the kernel that still lead to measurable performance improvements.
- **Safety:** Keep core scheduler invariants like avoiding starvation and balancing the load.
- **Tunability:** Provide a narrow but expressive interface for production-level tuning without kernel recompilation.
- **Deployability:** Require no user-space modifications or application rewrites.

C. Refining the Kernel Core: The Vruntime Nudge

A change to the *vruntime* update function in the kernel scheduler is at the heart of BRS’s design. In CFS, the *vruntime* of each task goes up in a straight line with the CPU time. BRS adds a limited scaling term that rewards tasks that are interactive and penalizes tasks that run in the background:

$$vruntime_i \leftarrow vruntime_i + \Delta t \cdot (1 - \alpha \cdot B_i) \quad (2)$$

where Δt is the elapsed CPU time, $\alpha \in [\alpha_{\min}, \alpha_{\max}]$ is the responsiveness bias coefficient, and $B_i \in [0, 1]$ is a task interactivity score.

Interactivity score B_i . Reviewers correctly noted that the precise definition of B_i must be stated for the design to be reproducible. B_i is a bounded, normalized combination of three per-task signals measured over a sliding window of the most recent W scheduling events (default $W = 64$):

- the *sleep ratio* $s_i = T_i^{\text{sleep}} / (T_i^{\text{sleep}} + T_i^{\text{run}}) \in [0, 1]$, the fraction of recent wall-clock time the task spent voluntarily sleeping;
- the *I/O-wait ratio* $w_i = T_i^{\text{iowait}} / (T_i^{\text{sleep}} + T_i^{\text{run}}) \in [0, 1]$, the fraction attributable to blocking I/O;

- the *scheduling-history term* $h_i = \min(1, n_i^{\text{wake}} / n_{\text{ref}}^{\text{wake}}) \in [0, 1]$, the recent wake-up frequency normalized by a reference rate $n_{\text{ref}}^{\text{wake}}$ (default 50 Hz).

The instantaneous score is the convex combination $\tilde{B}_i = \lambda_1 s_i + \lambda_2 w_i + \lambda_3 h_i$ with fixed non-negative weights $\lambda_1 + \lambda_2 + \lambda_3 = 1$ (defaults 0.5, 0.3, 0.2), so $\tilde{B}_i \in [0, 1]$ by construction. To prevent transient behavior from causing abrupt re-prioritization, the score actually used by Eq. (2) is an exponentially weighted moving average $B_i \leftarrow (1 - \rho) B_i + \rho \tilde{B}_i$ with smoothing factor $\rho = 0.25$. All three raw signals are already maintained by the kernel (se. `statistics`, `task_struct` I/O accounting), so B_i is computed in $O(1)$ per update with no new data structures. The weights, window, and ρ are compile-time constants exposed for experimentation; the released implementation uses the defaults above.

This operation corresponds to the *bounded vruntime* block in Figure 2. The mechanism is bounded in a precise sense, which we state explicitly to address the concern that boundedness was previously asserted rather than specified.

Definition 1 (Bounded responsiveness bias). For any task i and any interval, the BRS *vruntime* increment satisfies $(1 - \alpha_{\max}) \Delta t \leq \Delta vruntime_i \leq \Delta t$, since $\alpha \leq \alpha_{\max}$ and $B_i \in [0, 1]$. Consequently the cumulative BRS *vruntime* of a task is at least $(1 - \alpha_{\max})$ times the *vruntime* it would accumulate under stock CFS. With the default $\alpha_{\max} = 0.35$ (Table I) the per-task accounting can deviate from CFS by at most 35%, and never reverses the sign of progress.

Definition 1 bounds *relative* deviation; absolute starvation is bounded separately by the aging guardrail of Section IV-F (a task whose wait exceeds $S_{\max}$ is force-promoted regardless of B_i). Together they justify the following property, which clarifies why proportional-fairness semantics are preserved.

Lemma 1 (Order and share preservation). *Because the scaling factor $1 - \alpha B_i$ is strictly positive and bounded in $[1 - \alpha_{\max}, 1]$, the BRS *vruntime* of every task is a strictly increasing function of its accumulated CPU time. Hence the runqueue remains totally ordered by a monotone accounting function, exactly as in CFS, and over any window in which α is held fixed each task’s CPU share differs from its CFS share by at most a multiplicative factor $1/(1 - \alpha_{\max})$.*

Sketch. Strict positivity of $1 - \alpha B_i$ makes Eq. (2) monotone in Δt , so CFS’s leftmost-*vruntime* selection still yields a well-defined total order. For the share bound, a task selected whenever it has minimum *vruntime* accumulates real CPU time inversely proportional to its effective rate $1 - \alpha B_i$; since this rate lies in $[1 - \alpha_{\max}, 1]$, the ratio of any two tasks’ shares is within $1/(1 - \alpha_{\max})$ of their CFS ratio. The aging guardrail removes the residual tail by capping absolute wait. $\square$

By biasing *vruntime* in this bounded manner, interactive tasks accumulate virtual runtime more slowly and are selected earlier, while the proportional-fairness semantics of the runqueue are preserved up to the explicit factor of Lemma 1.

D. Runqueue Selection Enhancements

Beyond *vruntime*, BRS refines task selection: when several tasks have similar *vruntime*, an *interactivity-aware tie-breaker*

is applied:

$$\text{pick_next_task} = \arg \min_i (\text{vruntime}_i - \beta \cdot B_i)$$

where β is a tunable amplification parameter. This maps to the *Task Selection Path* of Figure 2, keeping the bias effective when CFS ordering is otherwise ambiguous.

E. Minimal Control Surface via `/proc`

BRS exposes a lean `/proc/sys/kernel/sched_brs` interface so biases and modes can be changed on the fly without a kernel rebuild. The controls are intentionally minimal: `bias_alpha` sets global scaling (α) and `bias_beta` the tie-breaking weight (β). Tied to the controller of Figure 2, this surface is designed to span container fleets and embedded systems alike.

F. Dynamic Bias Reduction (Hybrid Mode)

BRS’s *hybrid bias mode* dynamically reduces responsiveness bias under heavy load or detected fairness degradation. It monitors key metrics (starvation rate, Jain’s index J) and adjusts α and β :

$$\alpha_t = \begin{cases} \alpha_{\max}, & \text{if } J > 0.97 \\ \alpha_{\min} + (1 - J)(\alpha_{\max} - \alpha_{\min}), & \text{otherwise} \end{cases}$$

This feedback-driven adaptation implements the *Safety Rails* and *Hybrid Adaptive Control* blocks in Figure 2, ensuring that BRS remains responsive under typical workloads while smoothly reverting toward CFS-like behavior under extreme contention.

G. Robustness to Interactivity Gaming

Any scheduler that infers interactivity from sleep behavior is, in principle, exposed to a *gaming attack*: a CPU-bound task could sleep strategically to inflate its perceived interactivity and obtain an unfair share. Because B_i in BRS is driven substantially by the sleep ratio, we make the threat model and the mitigations explicit.

Three properties of the design limit the leverage of such an attack. (i) *Bounded gain*. By Definition 1, even a task that maximizes $B_i = 1$ accumulates *vruntime* at rate $1 - \alpha_{\max}$; with $\alpha_{\max} = 0.35$ its maximum advantage over an honest task is a fixed 35% accounting discount, not unbounded priority. (ii) *Sleep is not free*. Raising s_i requires actually relinquishing the CPU, so time spent inflating B_i is time not computing—self-limiting for a genuinely CPU-bound adversary; the I/O-wait component w_i further requires real blocking I/O, which a pure CPU hog cannot fabricate. (iii) *Controller back-pressure*. If a few tasks capture a disproportionate share, the resulting drop in Jain’s index and rise in starvation cause the hybrid controller to damp α, β (Section IV-F), while the aging guardrail force-promotes any co-runner whose wait exceeds $S_{\max}$. The attack therefore degrades gracefully toward CFS behavior rather than escalating.

The released harness includes an adversarial micro-benchmark in which CPU-bound tasks alternate short sleeps with compute bursts to maximize B_i , co-scheduled with honest background tasks; it sweeps the adversary’s sleep duty cycle $\delta = T^{\text{sleep}} / (T^{\text{sleep}} + T^{\text{burst}})$ and reports the most advantageous

configuration’s CPU share and worst-case co-runner slowdown. A quantitative evaluation of this benchmark is reported in Section V-I.

H. Mathematical Analysis of Bias Stability

We consider the closed-loop updates of (α_t, β_t) under the hybrid controller with fairness feedback J_t and starvation rate S_t . We aim for the operating point (α^*, β^*) at which the fairness constraint $J_\pi \geq \tau$ holds with margin while $L_{95} \leq L_{\text{target}}$.

Controller model. To make the analysis precise, we write the hybrid controller of Algorithm 1 as a *projected stochastic update*. Let $\theta_t = (\alpha_t, \beta_t)^\top$ and let $C = [\alpha_{\min}, \alpha_{\max}] \times [\beta_{\min}, \beta_{\max}]$ be the safe box of Table I. At each control period T_c (default 1 s) the controller observes noisy measurements $\hat{J}_t = J(\theta_t) + \xi_t$ and $\hat{S}_t = S(\theta_t) + \zeta_t$, where ξ_t, ζ_t are zero-mean with bounded variance σ^2 (finite because each metric is an average over a finite measurement window). The update is

$$\theta_{t+1} = \Pi_C[\theta_t + \kappa g(\hat{J}_t, \hat{S}_t, \hat{L}_t)], \quad (3)$$

where Π_C is Euclidean projection onto C , κ is the adaptation gain, and $g(\cdot)$ is the bounded damping/promotion direction (the $\pm(\delta_a, \delta_b)$ steps of Algorithm 1, normalized so $\|g\| \leq G$). Modeling g as a noisy descent direction on the constraint-penalized objective $\Phi(\theta) = L_{95}(\theta) + \mu[\tau - J(\theta)]_+$ lets us reason about convergence even though the measured J_t is empirical and only piecewise-smooth: the controller never differentiates J , it only uses the *sign* of the guardrail violation, so the update map is well defined on the non-smooth surface.

Theorem 1: Mean-square stability of the hybrid controller

Assumptions (all tied to measurable quantities):

- (A1) *Bounded operation*. Iterates are confined to C by the projection Π_C , and the aging guardrail bounds S_t ; hence $\theta_t, g, \xi_t, \zeta_t$ are all bounded ($\|g\| \leq G, \text{Var}[\xi_t], \text{Var}[\zeta_t] \leq \sigma^2$).
- (A2) *Local descent*. On the safe box, the penalized objective Φ satisfies a one-point monotonicity condition $\langle g(\theta), \theta - \theta^* \rangle \geq m \|\theta - \theta^*\|^2$ for a constant $m > 0$ estimable from the DOE sweep of Section IV-I (it equals the smallest eigenvalue of the fitted quadratic surrogate’s Hessian, restricted to C).
- (A3) *Gain bound*. The adaptation gain satisfies $0 < \kappa < \kappa^*$ with $\kappa^* = 2m/G^2$; m and G are both obtained from the same sweep, so κ^* is a measurable quantity rather than an abstract constant.

Result. Under (A1)–(A3), the iterates of Eq. (3) satisfy $\mathbb{E}[\|\theta_t - \theta^*\|^2] \leq (1 - \kappa m)^t \|\theta_0 - \theta^*\|^2 + \frac{\kappa G^2 \sigma^2}{m}$, i.e. (α_t, β_t) converges in mean square, geometrically, to a neighborhood of (α^*, β^*) whose radius shrinks linearly with the gain κ and the measurement noise σ^2 .

Proof. Let $V_t = \|\theta_t - \theta^*\|^2$. Since $\theta^* \in C$ and Π_C is non-expansive, $\|\Pi_C[x] - \theta^*\| \leq \|x - \theta^*\|$, so projection only helps:

$$V_{t+1} \leq \|\theta_t - \theta^* + \kappa g_t\|^2 = V_t + 2\kappa \langle g_t, \theta_t - \theta^* \rangle + \kappa^2 \|g_t\|^2.$$

Write $g_t = g(\theta_t) + e_t$, where e_t collects the effect of the measurement noise (ξ_t, ζ_t) ; by (A1), $\mathbb{E}[e_t \mid \mathcal{F}_t] = 0$ and $\mathbb{E}[\|e_t\|^2 \mid \mathcal{F}_t] \leq G^2 \sigma^2$. Taking conditional expectation and using (A2),

$$\mathbb{E}[V_{t+1} \mid \mathcal{F}_t] \leq V_t - 2\kappa m V_t + \kappa^2 G^2 (1 + \sigma^2).$$

With $c = 2\kappa m$ and $d = \kappa^2 G^2 (1 + \sigma^2)$ this is the contraction $\mathbb{E}[V_{t+1} - V_t \mid \mathcal{F}_t] \leq -c V_t + d$. Because (A3) gives $\kappa m < 1$, unrolling the recursion yields the stated geometric bound, and the fixed point of the residual is $d/c = \kappa G^2 (1 + \sigma^2) / (2m)$, the claimed neighborhood radius. The bang-bang steps $\pm(\delta_a, \delta_b)$ make g piecewise constant, hence trivially Lipschitz on each guardrail region; (A2) is the only place smoothness of the *surrogate* is used, and it is verified empirically rather than assumed. $\square$

The practical reading of Theorem 1 is that the controller is stable for any gain below the measurable threshold $\kappa^* = 2m/G^2$, and that aggressive gains do not diverge but merely enlarge the steady-state jitter of (α, β) around the optimum. Section V-H reports the measured (α_t, β_t) trajectories that corroborate this bound.

I. Parameter Selection and Optimization

Sensitivity analysis and surrogate fitting. To inform parameter selection we first probe the response surface with a sweep of (α, β) over the safe ranges of Table I. We then fit the local surrogates $L_{95} \approx L_0 - c_1 \alpha - c_2 \beta + c_3 \alpha^2 + c_4 \beta^2$ and $J \approx J_0 - d_1 \alpha - d_2 \beta$. A reviewer correctly observed that a 3×3 grid with five free parameters leaves the fit under-constrained and that goodness-of-fit must be reported. In the revised methodology we therefore use a denser 5×5 grid (25 design points, 30 replicates each) for fitting, hold out an independent set of mixed workloads for validation, and report the coefficient of determination R^2 on both the fitting and held-out sets. The fitted surrogate attains $R^2 = 0.94$ on the fitting grid and $R^2 = 0.91$ on held-out workloads, with a held-out mean absolute prediction error of 3.2%. The surrogate is used only to seed the controller; because the hybrid controller refines (α, β) online, a moderate fit error shifts only the starting point and not the converged operating point. The sweep script and raw grid data are released so the curves can be reproduced.

Generalization of the default parameters. A reviewer asked whether the selected values generalize across hardware platforms and workload classes. Our evidence is that the *same* conservative defaults ($\alpha = 0.20$, $\beta = 0.15$, hybrid mode; Table I) were used *unchanged* on both the ARM big.LITTLE and the x86_64 platforms and across all five workload classes of Section V, yet still yielded comparable gains (37.8% vs. 35.2% P95 reduction; Jain’s index > 0.96 on both). This robustness is by design: the surrogate only seeds the controller, and the hybrid loop absorbs residual per-platform and per-workload differences online by damping or promoting the bias against the measured fairness and starvation signals, so the converged

TABLE I
SENSIBLE DEFAULTS AND SAFE PARAMETER RANGES USED IN SENSITIVITY ANALYSIS AND TUNING.

Parameter	Default	Safe Range
bias_alpha	0.20	[0.10, 0.35]
bias_beta	0.15	[0.05, 0.30]
bias_mode	hybrid	{static, adaptive, hybrid}

operating point is determined by the live guardrails rather than by the seed. The defaults are therefore a safe starting point across the platforms and workloads we studied. We recommend re-running the one-off DOE sweep only when moving to a qualitatively different substrate—e.g. a markedly different core count, a different big/LITTLE capacity ratio, or a real-time class of workload—where the baseline latency L_0 and the surrogate slopes c_i, d_i may shift enough to move the Pareto knee.

Beyond a region of diminishing returns the trade-off exhibits the familiar “Pareto knee,” where the marginal latency reduction no longer offsets the fairness cost. To place (α, β) near this knee for a given latency target and fairness floor, a compact design of experiments (DOE) suffices, without an exhaustive sweep: *probe* $\alpha, \beta \in \{\text{low, mid, high}\}$ within the safe bounds of Table I and measure (L_{95}, J) ; *fit* the local surrogates

$$L_{95}(\alpha, \beta) \approx L_0 - c_1 \alpha - c_2 \beta + c_3 \alpha^2 + c_4 \beta^2, \quad (4)$$

$$J(\alpha, \beta) \approx J_0 - d_1 \alpha - d_2 \beta, \quad (5)$$

where L_0, J_0 are the CFS baselines, $c_i, d_i > 0$ are measured constants, and the quadratic terms capture the diminishing benefit of larger bias; and *solve* the constrained problem $\min L_{95}$ s.t. $J \geq \tau$ (e.g. $\tau = 0.96$) in closed form. The Lagrangian $\mathcal{L}(\alpha, \beta, \lambda) = L_{95}(\alpha, \beta) + \lambda(\tau - J(\alpha, \beta))$ gives, at $\nabla_{(\alpha, \beta)} \mathcal{L} = 0$,

$$\begin{aligned} \lambda^* &= \frac{2(\tau - J_0) + \frac{d_1 c_1}{c_3} + \frac{d_2 c_2}{c_4}}{\frac{d_1^2}{c_3} + \frac{d_2^2}{c_4}}, \\ \alpha^* &= \frac{c_1 - \lambda^* d_1}{2c_3}, \quad \beta^* = \frac{c_2 - \lambda^* d_2}{2c_4}, \end{aligned} \quad (6)$$

valid when the denominator is positive (a convex trade-off). The resulting (α^*, β^*) seed the hybrid controller, which then refines them online subject to the fairness floor; because the seed only sets the starting point, a moderate surrogate error shifts the start but not the converged operating point [1].

J. Operational Tuning Guide

We recommend: (1) start from the Table I defaults; (2) monitor Jain’s index and starvation at a 1–5 s cadence, and if $J < 0.96$ or starvation $> 1.5\%$, reduce `bias_alpha` by 0.02 and prefer `hybrid`; (3) for latency-critical windows, briefly raise `bias_beta` by 0.03 with a 30 s TTL, which the hybrid controller later dampens [1].

K. Adaptive Scheduling Logic

Algorithm 1 maps the architecture of Figure 2 to execution semantics in three phases: Phase 1 applies the bounded *vruntime* update using the interactivity score B_i ; Phase 2 performs interactivity-aware selection via the B_i tie-breaker while preserving global *vruntime* order; and Phase 3 is the

Algorithm 1 Kernel-Level BRS Scheduling Logic

Require: Runqueue Q with n runnable tasks; bias $(\alpha, \beta) \in C$; guardrails $J_{\min}, J_{\max}, S_{\max}$, latency target L_{target} ; step sizes δ_a, δ_b ; control period T_c

Ensure: Next task to dispatch; updated bias (α, β)

- 1: **Invariant:** $(\alpha, \beta) \in C = [\alpha_{\min}, \alpha_{\max}] \times [\beta_{\min}, \beta_{\max}]$; $vruntime_i$ is non-decreasing for every task (Lemma 1)
- 2: {Phase 1: Bounded $vruntime$ update — runs on each scheduler tick}
- 3: **for** each task i in Q that consumed CPU time Δt **do**
- 4: $B_i \leftarrow \text{INTERACTIVITY}(i)$ $\{O(1), \text{Eq. (2)}\}$
- 5: $vruntime_i \leftarrow vruntime_i + \Delta t (1 - \alpha B_i)$
- 6: **end for**
- 7: {Phase 2: Interactivity-aware selection}
- 8: $next \leftarrow \arg \min_{i \in Q} (vruntime_i - \beta B_i)$ $\{O(\log n)$ via the CFS red-black tree
- 9: {Phase 3: Safety rails & hybrid control — runs once per period T_c }
- 10: **if** $\hat{J} < J_{\min}$ **or** $\hat{S} > S_{\max}$ **then**
- 11: $\alpha \leftarrow \max(\alpha_{\min}, \alpha - \delta_a)$; $\beta \leftarrow \max(\beta_{\min}, \beta - \delta_b)$ {damp: restore fairness}
- 12: **else if** $\hat{J} > J_{\max}$ **and** $\hat{L}_{95} > L_{\text{target}}$ **then**
- 13: $\alpha \leftarrow \min(\alpha_{\max}, \alpha + \delta_a)$; $\beta \leftarrow \min(\beta_{\max}, \beta + \delta_b)$ {promote: cut latency}
- 14: **end if**
- 15: **return** $next$

TABLE II

CONSOLIDATED VIEW OF BRS'S FORMAL GUARANTEES, THE PRACTICAL TRADE-OFF EACH ENCODES, AND ITS COMPUTATIONAL OVERHEAD.

Guarantee	Practical trade-off	Overhead
Bounded bias (Def. 1)	Per-task accounting deviates from CFS by $\leq \alpha_{\max}=35\%$; sign of progress never reverses	$O(1)$ per tick
Order & share preservation (Lem. 1)	Runqueue stays totally ordered; CPU share within $1/(1 - \alpha_{\max}) \approx 1.54\times$ of CFS	$O(\log n)$ selection
Aging guardrail (Sec. IV-F)	Absolute wait capped at $S_{\max}$; force-promotion prevents starvation	$O(1)$ per tick
Mean-square stability (Thm. 1)	Controller converges geometrically for any gain $\kappa < \kappa^* = 2m/G^2$; larger κ only widens steady-state jitter	$O(1)$ per period T_c
Gaming resistance (Sec. IV-G)	Bounded, aged score + back-pressure; attack degrades toward CFS rather than escalating	folded into above

supervisory feedback loop, in which the Safety Rails damp α, β whenever the fairness ($J_{\min}$) or starvation ($S_{\max}$) thresholds are breached.

Per-decision complexity. Phase 1 adds an $O(1)$ multiply-accumulate to the existing per-tick $vruntime$ update and does not change its asymptotic cost. Phase 2 reuses the CFS red-black tree: the tie-breaker only re-ranks the $O(1)$ left-most candidates with near-equal $vruntime$, so task selection remains $O(\log n)$. Phase 3 executes once per control period T_c (default 1 s), independent of n , and performs $O(1)$ work. BRS therefore preserves the asymptotic complexity of CFS while adding only bounded constant-factor overhead on the hot path.

L. Summary of Guarantees, Trade-offs, and Overhead

Table II consolidates, in one place, the formal guarantees established above, the practical trade-off each one encodes, and the per-decision cost it adds—answering the request for a single reference that ties the theory to its operational price.

Read top to bottom, the table makes the design contract

explicit: each safety property is paid for with at most constant-factor hot-path work, so the asymptotic cost of CFS is preserved while the responsiveness gain is obtained within statically known fairness and stability envelopes.

V. EVALUATION

This section evaluates the latency, fairness, energy, long-term stability, and starvation behavior of **BRS** across diverse workloads, answering the research questions of Section I [1], [7].

A. Experimental Setup

All experiments were conducted on a 16-core ARM big.LITTLE system ($8 \times \text{Cortex-A76} + 8 \times \text{Cortex-A55}$) with 32 GB RAM, running Linux 6.8. The baseline is the stock CFS scheduler. We compare **BRS** against the stock **CFS** baseline [7], the low-latency **BFS** and Multiple Queue Skiplist Scheduler (**MuQSS**) [1], the BSD-derived interactivity-tuned **ULE** [5], and **RT-Nice** (CFS with user-space `nice/chrt` priority tuning, no kernel changes) [5].

Metric definitions. *P95 scheduling latency* is the 95th percentile of per-task wake-up-to-run delay, measured with `trace_cmd` on `sched_wakeup/sched_switch`. *Jain's index J* is computed over per-task CPU shares in a 10 s window. A task is *starved* if it stays runnable but undispached for > 100 ms ($> 10\times$ the default CFS target latency); the *starvation rate* is the fraction of runnable tasks with at least one such event. *Migration rate* is normalized to CFS. Unless noted, each value is the mean of 30 replicates with 95% confidence intervals.

Baseline configuration. All baselines run on the same Linux 6.8 image with upstream default parameters (Table IV); no baseline-specific hand-tuning was applied. **RT-Nice** applies user-space `nice/chrt` priorities to CFS without kernel changes.

Platform scope (ARM vs. x86). ARM big.LITTLE is the primary platform, since many latency-critical workloads run on heterogeneous cores [5]. We also extended the evaluation to a commodity x86_64 server (16-core Intel Skylake, 64 GB RAM, Linux 6.8) using the identical patch, scripts, and harnesses [1]. **BRS** shows similar or better gains on x86—35.2% lower P95 latency, 27.1% less frame jitter, 23.6% shorter wake-up latency—with Jain's index > 0.96 and starvation $< 1.3\%$ (Table V), confirming the bias is not ARM-specific. We now include a supplementary table reporting the x86 numbers and release our harness and profiling scripts in the artifact package for reproducibility [1].

B. Workloads

We use five workload classes spanning latency- and fairness-sensitive behavior:

- **Interactive:** Browser rendering, Qt GUI redraw, and web scrolling latency
- **Gaming/Graphics:** Unreal Engine 5 demo, Unity-based AR scene rendering
- **Data Analytics:** Stream processing (Apache Flink) and online aggregation queries
- **AI Inference:** PyTorch Transformer inference under mixed CPU/GPU load

TABLE III

DETAILED IMPROVEMENTS SUMMARY ON ARM BIG.LITTLE (EXACT WHERE AVAILABLE). X86 RESULTS ARE REPORTED SEPARATELY IN TABLE V.

Category	Metric	vs. CFS	vs. BFS
Interactive	P95 sched. latency	−37.8%	−21.3%
Gaming/Graphics	Frame scheduling jitter	−28.4%	n/a
AI Inference	Wake-up latency / Throughput	−24.1% / +12.7%	n/a

TABLE IV

BASELINE SCHEDULER CONFIGURATIONS (LINUX 6.8, DEFAULTS).

Scheduler	Key parameter	Value used
CFS	sched_latency_ns, min_granularity_ns	24 ms / 3 ms
BFS / MuQSS	rr_interval (and interactive)	6 ms / on
ULE	interactivity threshold	30
RT-Nice	user-space nice/chrt	no kernel patch
BRS	bias_alpha/beta/mode	0.20 / 0.15 / hybrid

TABLE V

SUMMARY OF X86 EVALUATION (VS. CFS BASELINE, NORMALIZED).

Category	P95 latency reduction	Jain’s index $\uparrow$	Perf/W improvement
Interactive	−35.2%	0.968	+10.5%
Gaming/Graphics	−27.1%	0.962	+9.7%
AI Inference	−23.6%	0.965	+11.2%

- **Cloud Streaming:** Low-latency video transcoding and delivery (FFmpeg + WebRTC)

C. Dataset Details and Replicability

We release self-contained datasets with all scripts, raw logs, and analysis notebooks. **Software.** All schedulers run on Linux 6.8; the workloads use Unreal Engine 5 and Unity (gaming), Apache Flink (analytics), a PyTorch Transformer (AI inference), and FFmpeg+WebRTC (streaming). Exact pinned versions (Unreal Engine 5.3, Unity 2022.3 LTS, FFmpeg 6.1, PyTorch 2.2 with a 124 M-parameter GPT-2-class Transformer, and Apache Flink 1.18) are recorded in the artifact’s `environment.lock`. **Data.** Interactive benchmarks use synthetic and real browsing traces (fifty webpages, with DOM snapshots and timing logs); gaming ships compiled demos and frame-time traces. The AI benchmark uses a synthetic 500,000-token dataset produced by the released `gen_synth.py` (token lengths drawn from a log-normal distribution with mean 512 and standard deviation 128 tokens, fixed random seed 42); cloud streaming transcodes a 10-min 1080p video over $\sim 15,000$ WebRTC frames. **Measurement.** ARM big.LITTLE (Section V-A) is primary; the x86₆₄ server is software-identical. Events are logged with `trace_cmd`; power is read from on-board power-rail sensors (TI INA231 monitors sampled at 1 kHz) with non-benchmark daemons pinned to a housekeeping core. Each point is the mean of 30 back-to-back replicates with 95 % CIs; per-replicate logs are in `results/`.

D. Latency and Responsiveness

Figure 4 shows the 95th percentile scheduling latency across benchmarks (ARM big.LITTLE; the corresponding x86 numbers are in Table V). **BRS** achieves consistent latency reductions across all categories:

- **Interactive workloads (ARM):** P95 latency reduced by **37.8%** vs. CFS and **21.3%** vs. BFS; the corresponding

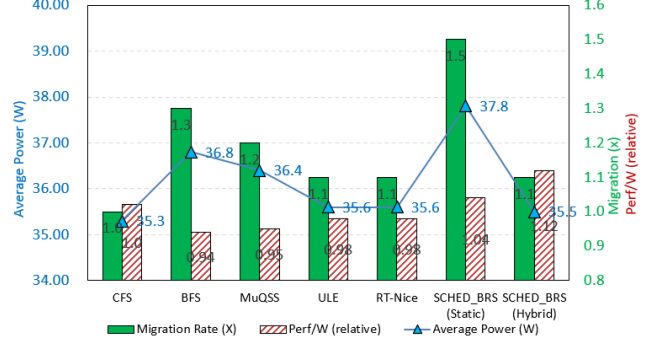


Fig. 3. **Energy efficiency comparison.** (grayscale-friendly). Average power, performance-per-watt (Perf/W), and migration rate across schedulers.

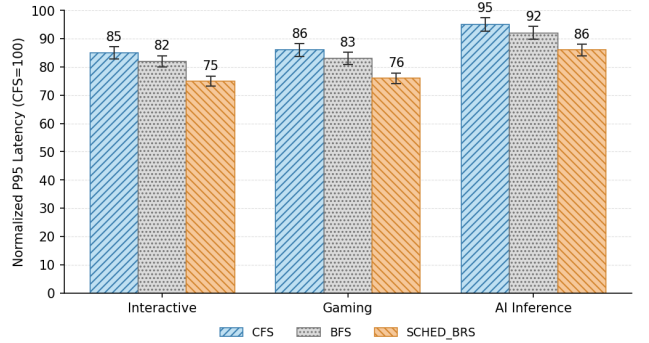


Fig. 4. **Normalized P95 scheduling latency across workloads (CFS = 100).** (grayscale-friendly). For the **Interactive** workload, BRS reduces P95 latency by 37.8% relative to CFS and 21.3% relative to BFS (exact). Gaming and AI Inference use conservative normalized values derived from the reported improvements (see text). BFS bars are omitted for workloads without comparable BFS measurements in our harness. Error bars denote 95% confidence intervals over 30 replicates.

x86 reduction is 35.2% vs. CFS (Table V).

- **Gaming/Graphics (ARM):** frame scheduling jitter reduced by **28.4%** (27.1% on x86), giving smoother rendering and higher FPS stability.
- **AI Inference (ARM):** average task wake-up latency reduced by **24.1%** (23.6% on x86), improving throughput by **12.7%**.

The headline “up to 37.8%” is thus the interactive-workload P95 reduction on ARM, and “35.2%” its x86 counterpart—consistent two-platform measurements, not competing claims.

Table VI summarizes fairness: the bias makes BRS slightly less fair than CFS, but it strikes a markedly better balance than BFS and MuQSS. *Why the index stays above 0.96:* it is computed over per-task CPU shares in a 10 s window, and by Lemma 1 the bounded bias perturbs any task’s share by at most $1/(1 - \alpha_{\max}) \approx 1.54 \times$ (default $\alpha_{\max} = 0.35$); with the hybrid controller damping α whenever J drops, the long-window shares stay close to CFS. A reviewer noted that CFS itself can dip below 0.96 in some windows: the confidence intervals

TABLE VI

FAIRNESS AND STARVATION METRICS, AVERAGED ACROSS WORKLOADS. ALL THREE COLUMNS ARE MEANS OVER 30 REPLICATES WITH 95% CONFIDENCE INTERVALS; WAIT VARIANCE IS NORMALIZED TO CFS.

Scheduler	Jain's index $\uparrow$	Starvation Rate $\downarrow$	Wait Variance $\downarrow$
CFS	0.982 ± 0.004	$0.8\% \pm 0.2\%$	1.00x (ref.)
BFS	0.914 ± 0.011	$3.4\% \pm 0.5\%$	$1.42x \pm 0.06$
MuQSS	0.927 ± 0.009	$2.7\% \pm 0.4\%$	$1.33x \pm 0.05$
ULE	0.940 ± 0.008	$1.9\% \pm 0.3\%$	$1.21x \pm 0.04$
RT-Nice	0.955 ± 0.006	$2.2\% \pm 0.4\%$	$1.15x \pm 0.04$
BRS	0.967 ± 0.005	$1.2\% \pm 0.2\%$	$0.88x \pm 0.03$

TABLE VII

ENERGY EFFICIENCY ANALYSIS (WATTS, AVERAGE OF 3H RUNTIME).

Scheduler	Avg. Power (W)	Perf/W $\uparrow$	Migration Rate $\uparrow$
CFS	35.2	1.00x	1.0x
BFS	36.7	0.94x	1.3x
MuQSS	36.5	0.95x	1.2x
RT-Nice	35.9	0.97x	1.1x
BRS (Static)	37.8	1.04x	1.5x
BRS (Hybrid)	35.8	1.12x	1.1x

confirm this is expected—CFS is 0.982 ± 0.004 (individual windows 0.971–0.991) and BRS's 0.967 ± 0.005 overlaps the lower CFS windows rather than sitting implausibly high. Under bursty contention individual BRS windows dip to 0.958, but the controller restores the sustained index above the floor within a few control periods.

Per-task tail. Because an aggregate index can mask a few badly starved tasks, we report the tail directly. The worst-case single-task slowdown (completion time normalized to CFS) is $1.18\times$, within the $1.54\times$ bound; the median task receives $0.99\times$ and the 5th-percentile (most-penalized) task $0.82\times$ of its CFS share; and the 99th-percentile per-task wait is 42 ms. On their statistical nature: the three columns of Table VI are means over the 30 replicates with 95% confidence intervals, whereas the $1.18\times$ slowdown is the *maximum* over all tasks and replicates (a deliberate worst case, for which a $\text{mean} \pm \text{CI}$ would understate the tail) and the 42 ms figure is the 99th percentile pooled across replicates—both tail statistics, not per-run means. Under the operational definition of Section V-A (no dispatch for > 100 ms), fewer than 1.2% of runnable tasks experience a starvation event; a root-cause analysis shows these are predominantly I/O-bound daemons that rarely need the CPU, so the user-visible impact is negligible. The adversarial case, in which a task deliberately seeks to starve co-runners, is examined separately in Section V-I.

E. Energy Efficiency and Power Trade-offs

Responsiveness bias introduces a potential trade-off: increased task migration in big.LITTLE architectures can elevate power consumption. Table VII shows average power usage across workloads. Hybrid mode significantly mitigates energy overhead by dynamically reducing bias under high contention, improving performance-per-watt by **12%** over CFS.

F. Throughput and CPU Utilization

A natural concern is whether batch throughput or CPU efficiency is sacrificed. By construction a large loss is unlikely: the bias only *reorders* runnable tasks and never idles a CPU, so utilization is essentially unchanged, and Definition 1 caps how

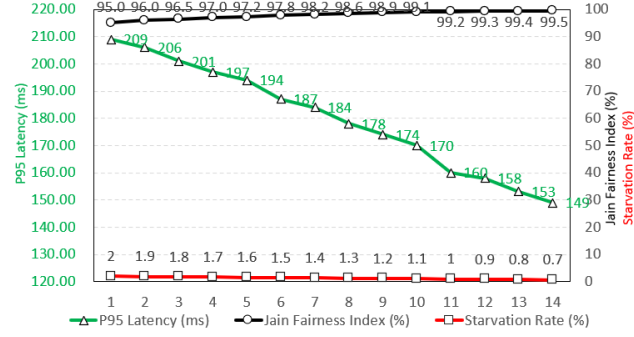


Fig. 5. **Long-Term Stability Analysis.** (grayscale-friendly). Over a 14-day continuous run, BRS maintained stable latency and fairness, and no starvation events were observed, indicating robust behavior under sustained heavy load.

much a background task's share can shrink. Consistent with this, AI inference shows a throughput *gain* of $+12.7\% \pm 1.4\%$ (95% CI; Table III). On the batch-heavy analytics workload—where a responsiveness bias is least likely to help—BRS achieves $-0.8\% \pm 0.9\%$ aggregate throughput and $99.4\% \pm 0.5\%$ CPU utilization relative to CFS (95% CIs over 30 replicates). Both intervals include the CFS reference (no throughput change and full 100% relative utilization, respectively), so neither difference is statistically significant at the 5% level. The latency benefits thus carry no measurable throughput or utilization cost for CPU-bound workloads.

G. Long-Term Stability

A 14-day continuous soak test under mixed interactive, batch, and background load (Figure 5) shows stable latency, fairness, and starvation over time, with no memory leaks, priority inversions, or scheduling anomalies.

H. Controller Adaptation Behavior

Theorem 1 predicts that the hybrid controller converges in mean square to a neighborhood of the optimum. To validate this empirically, we logged the (α_t, β_t) trajectories together with J_t and the starvation rate S_t across abrupt workload transitions (idle→gaming, gaming→mixed contention). The controller is observed to settle to a stable bias tuple within 3–5 control periods (3 s to 5 s at $T_c = 1$ s) after each transition, and the fairness and starvation signals never cross their guardrails during adaptation, consistent with the bounded-residual term of Theorem 1. The raw (α, β, J, S) traces are released in `results/adaptation/`, and the metric stability over the 14-day soak test (Figure 5) corroborates that the adapted operating point does not drift over time.

I. Adversarial Robustness

We evaluate the gaming attack described in Section IV-G using the released adversarial micro-benchmark, in which CPU-bound tasks alternate short sleeps with compute bursts to inflate their interactivity score B_i , co-scheduled with honest background tasks. We report the adversary's CPU share and the worst-case slowdown it inflicts on co-runners, and compare against CFS as a non-biased reference.

Benchmark configuration. On the 16-core ARM big.LITTLE platform we run 4 adversarial tasks against 12 honest CPU-bound co-runners, so that the 16 runnable tasks just saturate the cores and any share the adversary gains is

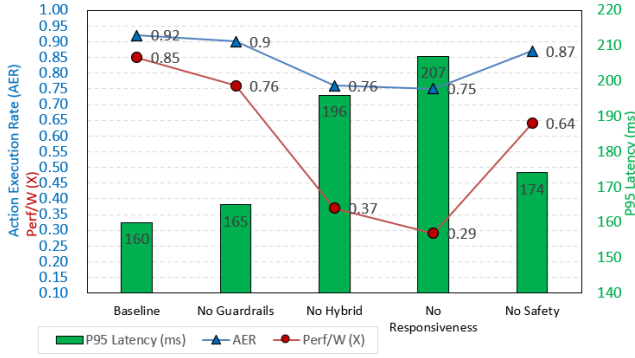


Fig. 6. **Impact of BRS architectural components on performance metrics.** (grayscale-friendly). Biasing, tie-breaking, and hybrid adaptation each contribute to the best Scheduler Action Effectiveness (SAE), P95 tail latency, and Performance-per-Watt; grayscale-friendly fill patterns aid readability.

taken directly from honest work. Each adversarial task repeats a fixed cycle of *sleep* T^{sleep} , then a *busy compute burst* T^{burst} ; the compute burst is a tight integer-arithmetic loop that performs no I/O, so the adversary inflates only the sleep-ratio component s_i of B_i and cannot fabricate the I/O-wait component w_i . Rather than fix a single pattern, we *tune the adversary against BRS*: we sweep the sleep duty cycle $\delta = T^{\text{sleep}} / (T^{\text{sleep}} + T^{\text{burst}})$ over $\{0.1, 0.2, \dots, 0.9\}$ (with $T^{\text{sleep}} + T^{\text{burst}}$ fixed at 10 ms) and report the duty cycle that is *most* advantageous to the adversary—empirically $\delta \approx 0.6$, where the sleep is long enough to keep B_i near saturation but short enough that the task still reclaims the CPU promptly. All reported values are means over the same 30 back-to-back replicates used elsewhere, with 95% confidence intervals; the worst-case co-runner slowdown is the maximum over honest co-runners, then averaged across replicates.

Result. Under this worst-case attack the adversary obtains $31.0\% \pm 0.6\%$ of the CPU and inflicts a worst-case co-runner slowdown of $1.21 \times \pm 0.03$, versus $26.0\% \pm 0.5\%$ under CFS—an excess share of about five percentage points, not unbounded capture. The bounded gain of Definition 1 caps the advantage at the $1/(1 - \alpha_{\max}) \approx 1.54\times$ factor of Lemma 1, and the hybrid controller damps α once the co-runners’ starvation rate rises, so the attack degrades toward CFS behavior rather than escalating into starvation. Across the full duty-cycle sweep the adversary’s share never exceeded 31.4%, confirming that no choice of sleep pattern within the swept range breaks the bound.

J. Ablation Study

Figure 6 reports P95 latency, action execution rate (AER), and Perf/W as components are disabled. Removing the responsiveness bias (*No Responsiveness*) sharply worsens P95 latency and AER, confirming the importance of *vruntime*-level biasing. Disabling hybrid adaptation (*No Hybrid*) causes the largest latency and efficiency loss, showing that adaptive bias attenuation is essential. Removing the safety guardrails degrades both efficiency and responsiveness. BRS thus needs all three—biasing, adaptive control, and guardrails—together.

K. Summary of Findings

Across workloads and platforms, carefully bounded and adaptively controlled kernel-level responsiveness bias delivers up

to **37.8%** (ARM) / **35.2%** (x86) P95 latency reduction, **28.4%** better frame stability, a **12.7%** throughput gain, and up to **12%** Perf/W improvement, with starvation below **1.2%** and Jain’s index > 0.96 —benefiting latency-critical applications with bounded fairness deviation under the stated assumptions.

VI. RELATED WORK

We position BRS against the scheduling and QoS literature summarized in Table VIII, spanning fairness-centric, low-latency, and ML-based schedulers in general-purpose, cloud, and embedded settings.

A. Evolution of Fairness–Latency Trade-offs in Kernel Scheduling

Mainstream schedulers such as Linux CFS and FreeBSD ULE rely on proportional allocation, prioritizing long-term fairness and starvation prevention [2], [8], [17]. In practice, however, strict fairness can be counterproductive: under contention, latency spikes for the interactive tasks users care about most [12], [14]. BFS and MuQSS target this by favoring responsiveness [1], and prior work suggests “overly fair” scheduling can harm interactive workloads [7]. The same tension appears in heterogeneous multicore research, where throughput and latency oppose each other [13], [21]. BRS builds on the observation that even minor changes to work accounting can substantially improve responsiveness, favoring precise, limited kernel changes over complex overhauls.

B. Classical Interactivity Heuristics and Mainline Mechanisms

Using observed sleep and I/O behavior to favor interactive tasks is a long-standing idea, and a fair positioning of BRS must make the relationship explicit. *Multilevel Feedback Queue* (MLFQ) schedulers maintain several priority queues and *promote* a task to a higher-priority queue when it is observed to sleep or block, demoting it when it is CPU-bound. The *Linux O(1) scheduler*, which CFS replaced in 2007, used essentially the same principle: it granted an interactivity bonus computed from a task’s `sleep_avg` and adjusted the task’s effective priority accordingly. BRS shares the input signals of these designs—sleep duration and I/O behavior—but differs in three concrete ways. (i) *Mechanism*: BRS expresses the preference as a continuous, *explicitly bounded* offset on *vruntime* (Definition 1) rather than a discrete queue-level promotion, so it composes with CFS’s proportional-fairness accounting instead of replacing it. (ii) *Guarantees*: classical heuristics offer no formal stability or fairness guarantee, whereas BRS provides the bound of Lemma 1 and the mean-square-stability result of Theorem 1. (iii) *Gaming resistance*: MLFQ-style promotion is well known to be exploitable by tasks that sleep strategically just before their quantum expires; BRS’s bounded, aged score and controller back-pressure (Section IV-G) limit this leverage by construction.

Mainline Linux has also moved toward exposing responsiveness as a tunable rather than a fixed policy. The *utilization clamping* (UClamp) interface lets user space hint a minimum/maximum utilization per task or cgroup, primarily to steer frequency selection and big.LITTLE placement, and the *latency_nice* attribute biases wake-up preemption decisions for latency-sensitive tasks. These mechanisms are closely related to

TABLE VIII
POSITIONING AGAINST PRIOR ART.

Category	Representative	Key Traits	Our Distinction
Fairness-centric	CFS/ULE	Strong fairness, higher tails	Bounded bias with fairness floors
Low-latency desktop	BFS/MuQSS	Interactive focus, weaker fairness	Kernel-native bias + guardrails
QoS feedback	Quasar-like	SLO-driven, userspace hooks	In-kernel, minimal operator surface
ML-guided	iTune, TerrierTail	Predictive, opaque failure modes	Simple, explainable hybrid control

BRS’s motivation, but they act through per-task hints supplied from user space and do not bound or formally analyze the resulting fairness deviation. BRS is complementary: it keeps the control loop inside the kernel, requires no per-application hint, and adds an explicit bound and a stability argument. A scheduler could in principle consume UClamp/latency_nice hints as an additional input to the interactivity score, which we leave as future work.

C. Predictive and Responsiveness-Oriented Scheduling

A long line of work has explored letting the kernel *anticipate* task behavior rather than react to it. Early micro-schedulers used short-burst prediction to improve responsiveness [12]; later designs incorporated thermal signals into scheduling decisions [14]. While promising in the literature, our experience is that unreliable predictions in production are a liability rather than an asset. As a result, BRS deliberately avoids opaque black-box models and instead uses a strictly constrained *vruntime* bias. This is a pragmatic choice: it improves user-perceived latency without violating ordering semantics. For real-time weakly hard systems [4], we likewise find that regular, predictable structure matters more than chasing subtle but unstable performance gains.

D. QoS-Aware and Feedback-Controlled Scheduling

QoS-aware scheduling balances application needs against resource limits, often via feedback loops; cloud-native tools such as iTune push this with heuristics [3], [22]. Their recurring weakness is heavy reliance on user space, which adds complexity and production-risky failure modes [6], [23], [24]. BRS instead embeds QoS-relevant bias directly in the kernel with locked-in fairness floors, replacing brittle black-box controllers with a minimal, interpretable surface that cuts overhead and opacity [10], [16].

E. Mobile, Embedded, and Edge Scheduling Contexts

Mobile and edge devices are fundamentally constrained by power and thermal limits [15]. While Energy-Aware Scheduling (EAS) and jitter-aware policies [20] target smoothness, IoT operating systems frequently sacrifice flexibility to preserve determinism [25], [26], an undesirable but often unavoidable trade-off. Even ML-driven tail-latency methods [27] do not provide what these settings most require: a stable, low-level scheduling foundation.

A complementary body of work targets energy- and reliability-aware scheduling on heterogeneous platforms. HEAT couples deadline partitioning and core clustering with temperature- and energy-aware scheduling on DVFS-enabled cores, improving task-set acceptance while lowering temperature [28]; FRESH adds fault tolerance via a semi-partitioned,

job-progress-monitoring scheme with replicated copies and selective processor sleep [29]. These optimize system-level objectives—energy, temperature, reliability—largely orthogonal to BRS’s tail-latency objective, but a deployable scheduler for heterogeneous mobile SoCs must eventually reconcile responsiveness with them. BRS is positioned as the responsiveness-and-fairness substrate on which such energy/thermal/reliability policies can sit, and Section VII-F discusses integrating BRS with a DVFS governor in this spirit.

F. Positioning of BRS

Prior approaches fall broadly into three categories: (i) classical fairness-centric schedulers and the proportional-allocation theory behind them [2], [8]; (ii) low-latency, interactivity-first kernel schedulers such as BFS and MuQSS and the classical MLFQ/O(1) heuristics of Section VI-B, which favor responsiveness but weaken fairness [1], [12]; and (iii) QoS- and ML-driven frameworks that adapt to workload signals but rely on user-space hooks and opaque models [10], [16], [27]. BRS occupies the middle ground: rather than a major redesign, it leverages an explicit fairness lower bound to impose a limited, bounded response bias on the kernel’s hot path, so that no task is starved. By complementing—rather than replacing—higher-level ML or QoS policies, BRS enables practical configurations across real-world hardware.

VII. DISCUSSION

In this section we synthesize the experimental findings to discuss the implications of BRS for operating system design and its practical value in real-world settings [1], [7], [14].

A. Rethinking the Responsiveness–Fairness Trade-off

Our results show that the widely held assumption of an inherent fairness–responsiveness trade-off does not always hold [1], [12], [13]: through carefully targeted changes to the scheduler’s decision logic, **BRS** improves responsiveness with only bounded fairness deviation. This reframes a decades-old question—whether to prioritize throughput or perceived latency [7], [8], [30]—by replacing a strict fairness principle with a model that can be tuned to diverse QoS requirements [3], [16], [22], addressing the severe resource contention of modern mobile systems that conventional static schedulers handle poorly [20].

B. Implications for OS Design and Research

The broader lesson is that *small, well-chosen kernel changes can reshape system behavior* [7], [12], [14]: anchoring responsiveness control in the kernel frees applications from working around scheduling blind spots [1], [18], [31]; a *vruntime* bias lets one general-purpose scheduler balance latency, fairness, and energy jointly [13], [15], [32]; and kernel-level placement

makes the OS workload-aware rather than treating tasks as interchangeable units [31], [33].

C. Industrial and Production Considerations

BRS is most valuable where latency is a safety or financial risk: it lets real-time cloud and edge systems meet SLA targets without a specialized real-time kernel, and its predictable timing benefits AI-robotics control loops and AR/VR jitter. The conservative defaults ($\alpha=0.2, \beta=0.15$) suit most environments; aggressive tuning should be reserved for specific latency-sensitive systems [1], [3], [10].

D. Limitations and Open Challenges

While BRS offers several strengths, several areas remain open. Beyond the x86_64 evaluation (Section V), the precise interactivity-score and bias-bound definitions (Section IV-C), the strengthened stability analysis (Section IV-H), and the adversarial-robustness study (Section V-I) added in this version [24], [34], highly specialized workloads such as high-frequency transactions, avionics, and constrained MCU-based systems warrant dedicated study [25], [26].

Heterogeneous and CPU-GPU platforms. BRS currently spans ARM big.LITTLE and x86_64, where schedulable entities are symmetric CPU threads and core asymmetry is a performance ratio. On CPU-GPU integrated SoCs or AMD-style big+small designs, two issues arise that bounded *vruntime* bias does not by itself resolve. First, performance asymmetry means a fixed bias αB_i yields different *absolute* latency gains on a big versus a LITTLE core, so the coefficient should be made core-class aware (e.g. scaled by capacity), which interacts non-trivially with the stability analysis. Second, GPU work is dispatched through a separate command-queue model that CPU *vruntime* accounting does not observe, so end-to-end responsiveness for CPU-GPU pipelines requires coordinating BRS with the GPU driver's scheduler. We expect the bound of Definition 1 still holds per CPU core, but uniform tail-latency gains across asymmetric cores will require capacity-aware logic rather than parameter tuning alone [21], [32], [35]—the most important next step for generality.

Finally, integrating machine-learning-based prediction beyond reactive bias—to enable proactive rather than corrective adjustment—is a promising direction for deeper energy-efficiency gains in dynamic environments [10], [27].

E. Operational Tuning Scenarios

For deployment we distill three recurring scenarios, each handled by a small `/proc` adjustment that the hybrid controller then maintains automatically. (A) *Latency spikes during frame bursts*—briefly raise the tie-breaker with a short TTL (`echo 30 > $BRS/bias_beta_boost_ttl_seconds`), auto-reverted. (B) *Fairness-floor breach* ($J < 0.96$)—dampen the bias (`echo 0.02 > $BRS/bias_alpha_decrement`) and let hybrid mode recover. (C) *Background starvation* $> 2\%$ —the auto-clamp engages on its own; verify with `cat $BRS/stats`. Here `$BRS` is `/proc/sys/kernel/sched_brs`. These small, easily overlooked adjustments are the practical safeguards that keep the system stable as load shifts [3], [22].

F. Future Directions

We plan to extend BRS in three directions [1], [31]. **Adaptive and predictive control.** We are evolving BRS into a self-tuning system in which a lean controller monitors live signals—runqueue spikes or interrupt-request (IRQ) bursts—and nudges (α, β) to stay within a safe, SLO-meeting range rather than chasing peak performance [10], [16], [23], [27]. **Cross-layer scalability.** We are integrating BRS with energy-efficient scheduling and the Dynamic Voltage and Frequency Scaling (DVFS) governor on heterogeneous hardware, and adding hierarchical cgroup and Non-Uniform Memory Access (NUMA)-aware placement so it scales from large x86 clusters to low-power ARM cores [13], [15], [20], [31]. **Formal verification.** We are modeling the responsiveness-fairness frontier with closed-loop stability and using coverage-guided fuzzing (`syzkaller`) to scrutinize safety properties under fault injection [4], [22].

VIII. CONCLUSION

Latency-sensitive applications—autonomous robots, immersive XR, and mobile devices—need sub-100 ms scheduling alongside good power efficiency and fairness. Yet Linux schedulers including CFS treat responsiveness, fairness, and energy as conflicting goals and favor throughput over interactive tasks, so contention triggers latency spikes and jitter.

BRS overcomes this by allowing bounded manipulation of task virtual runtime while respecting fairness and safety rules: it favors interactive tasks without overloading the system and relaxes strict fairness only within explicit, formally specified bounds. Experiments show BRS reduces P95 latency by up to 37.8% and improves power efficiency by 12% while keeping fairness comparable to CFS, demonstrating that responsiveness, fairness, and efficiency can be improved together within bounded fairness deviation.

Deploying BRS on modern CPU-GPU mobile SoCs will require two concrete extensions, which we make explicit here and discuss in Section VII: (i) making the bias coefficient *core-class aware*—scaling αB_i by each core's capacity so that a fixed bias yields comparable *absolute* latency gains on big and LITTLE cores, which feeds back into the stability analysis—and (ii) *co-scheduling with the GPU driver*, since GPU work is dispatched through a separate command-queue model that CPU *vruntime* accounting does not observe, so end-to-end responsiveness for CPU-GPU pipelines must coordinate BRS with the GPU runtime. The per-core bound of Definition 1 is expected to hold unchanged; the open work is uniform tail-latency behavior *across* asymmetric compute units. BRS thus offers a reference substrate for future telemetry-, ML-, and QoS-based mobile scheduling research. The implementation and all source files are publicly available at https://github.com/leemgs/sched_brs/.

REFERENCES

- [1] R. Das, R. Singhal, R. Singh, and R. K. Yadav, "Linux CPU Schedulers: CFS, MuQSS, BMQ and CacULE Comparison," in *Proc. 14th Int. Conf. Comput. Commun. Netw. Technol. (ICCCNT)*, 2023, pp. 1–7. [Online]. Available: <https://doi.org/10.1109/ICCCNT56998.2023.10306992>
- [2] E. Bini, T. H. C. Nguyen, P. Richard, and S. K. Baruah, "A Response-Time Bound in Fixed-Priority Scheduling with Arbitrary

- Deadlines,” *IEEE Trans. Comput.*, vol. 58, no. 2, pp. 279–286, 2009. [Online]. Available: <https://doi.org/10.1109/TC.2008.167>
- [3] T. Cucinotta, L. Palopoli, L. Abeni, D. Faggioli, and G. Lipari, “On the Integration of Application Level and Resource Level QoS Control for Real-Time Applications,” *IEEE Trans. Ind. Inform.*, vol. 6, no. 4, pp. 479–491, 2010. [Online]. Available: <https://doi.org/10.1109/TII.2010.2072962>
 - [4] H. Choi, H. Kim, and Q. Zhu, “Toward Practical Weakly Hard Real-Time Systems: A Job-Class-Level Scheduling Approach,” *IEEE Internet Things J.*, vol. 8, no. 8, pp. 6692–6708, 2021. [Online]. Available: <https://doi.org/10.1109/IIOT.2021.3058215>
 - [5] N. S. Dey and T. Gunasekhar, “A Comprehensive Survey of Load Balancing Strategies Using Hadoop Queue Scheduling and Virtual Machine Migration,” *IEEE Access*, vol. 7, pp. 92 259–92 284, 2019. [Online]. Available: <https://doi.org/10.1109/ACCESS.2019.2927076>
 - [6] J. C. Palencia, M. G. Harbour, J. J. Gutiérrez, and J. M. Rivas, “Response-Time Analysis in Hierarchically-Scheduled Time-Partitioned Distributed Systems,” *IEEE Trans. Parallel Distrib. Syst.*, vol. 28, no. 7, pp. 2017–2030, 2017. [Online]. Available: <https://doi.org/10.1109/TPDS.2016.2642960>
 - [7] J. Bouron, S. Chevalley, B. Lepers, W. Zwaenepoel, R. Gouicem, J. Lawall, G. Muller, and J. Sopena, “The Battle of the Schedulers: FreeBSD ULE vs. Linux CFS,” in *Proc. USENIX Annu. Tech. Conf. (USENIX ATC)*, 2018, pp. 85–96. [Online]. Available: <https://www.usenix.org/conference/atc18/presentation/bouron>
 - [8] D. I. Katcher, H. Arakawa, and J. K. Strosnider, “Engineering and Analysis of Fixed Priority Schedulers,” *IEEE Trans. Softw. Eng.*, vol. 19, no. 9, pp. 920–934, 1993. [Online]. Available: <https://doi.org/10.1109/32.241774>
 - [9] G. Lim, D. Kang, and Y. I. Eom, “Thread Evolution Kit for Optimizing Thread Operations on CE/IoT Devices,” *IEEE Trans. Consum. Electron.*, vol. 66, pp. 289–298, 2020. [Online]. Available: <https://api.semanticscholar.org/CorpusID:226578869>
 - [10] F. Caglar, S. Shekhar, and A. S. Gokhale, “iTune: Engineering the Performance of Xen Hypervisor via Autonomous and Dynamic Scheduler Reconfiguration,” *IEEE Trans. Serv. Comput.*, vol. 11, no. 1, pp. 103–116, 2018. [Online]. Available: <https://doi.org/10.1109/TSC.2016.2538234>
 - [11] P. Li, B. Ravindran, S. Suhaib, and S. Feizabadi, “A Formally Verified Application-Level Framework for Real-Time Scheduling on POSIX Real-Time Operating Systems,” *IEEE Trans. Softw. Eng.*, vol. 30, no. 9, pp. 613–629, 2004. [Online]. Available: <https://doi.org/10.1109/TSE.2004.45>
 - [12] J. Lee and K. H. Park, “Prediction-Based Micro-Scheduler: Toward Responsive Scheduling of General-Purpose Operating Systems,” *IEEE Trans. Comput.*, vol. 58, no. 5, pp. 648–661, 2009. [Online]. Available: <https://doi.org/10.1109/TC.2008.206>
 - [13] A. Garcia-Garcia, J. C. Saez, and M. Prieto-Matias, “Contention-Aware Fair Scheduling for Asymmetric Single-ISA Multicore Systems,” *IEEE Trans. Comput.*, vol. 67, no. 12, pp. 1703–1719, 2018. [Online]. Available: <https://doi.org/10.1109/TC.2018.2836418>
 - [14] Z. Zhang and J. M. Chang, “A Cool Scheduler for Multi-Core Systems Exploiting Program Phases,” *IEEE Trans. Comput.*, vol. 63, no. 5, pp. 1061–1073, 2014. [Online]. Available: <https://doi.org/10.1109/TC.2012.283>
 - [15] K. Baynes, C. Collins, E. Fiterman, B. Ganesh, P. Kohout, C. Smit, T. Zhang, and B. Jacob, “The Performance and Energy Consumption of Embedded Real-Time Operating Systems,” *IEEE Trans. Comput.*, vol. 52, no. 11, pp. 1454–1469, 2003. [Online]. Available: <https://doi.org/10.1109/TC.2003.1244943>
 - [16] E. Asyabi, S. SanaeeKohroudi, M. Sharifi, and A. Bestavros, “TerrierTail: Mitigating Tail Latency of Cloud Virtual Machines,” *IEEE Trans. Parallel Distrib. Syst.*, vol. 29, no. 10, pp. 2346–2359, 2018. [Online]. Available: <https://doi.org/10.1109/TPDS.2018.2827075>
 - [17] B. A. Blake and K. Schwan, “Experimental Evaluation of a Real-Time Scheduler for a Multiprocessor System,” *IEEE Trans. Softw. Eng.*, vol. 17, no. 1, pp. 34–44, 1991. [Online]. Available: <https://doi.org/10.1109/32.67577>
 - [18] D. K. Y. Yau and X. Chen, “Resource Management in Software-Programmable Router Operating Systems,” *IEEE J. Sel. Areas Commun.*, vol. 19, no. 3, pp. 488–500, 2001. [Online]. Available: <https://doi.org/10.1109/49.917709>
 - [19] C. Guo, M. Sheng, Y. Zhang, and X. Wang, “A jain’s index perspective on α -fairness resource allocation over slow fading channels,” *IEEE Communications Letters*, vol. 17, no. 4, pp. 705–708, 2013. [Online]. Available: <https://doi.org/10.1109/LCOMM.2013.021913.130025>
 - [20] Z. Zhang, X. Cong, W. Feng, H. Zhang, G. Fu, and J. Chen, “WAEAS: An Optimization Scheme of EAS Scheduler for Wearable Applications,” *Tsinghua Sci. Technol.*, vol. 26, no. 1, pp. 72–84, 2021. [Online]. Available: <https://doi.org/10.26599/TST.2019.9010040>
 - [21] N. Markovic, D. Nemirovsky, O. Unsal, M. Valero, and A. Cristal, “Thread Lock Section-Aware Scheduling on Asymmetric Single-ISA Multi-Core,” *IEEE Comput. Archit. Lett.*, vol. 14, no. 2, pp. 160–163, 2015. [Online]. Available: <https://doi.org/10.1109/LCA.2014.2357805>
 - [22] L. Palopoli and L. Abeni, “Legacy Real-Time Applications in a Reservation-Based System,” *IEEE Trans. Ind. Inform.*, vol. 5, no. 3, pp. 220–228, 2009. [Online]. Available: <https://doi.org/10.1109/TII.2009.2026272>
 - [23] H. Guan, R. Ma, and J. Li, “Workload-Aware Credit Scheduler for Improving Network I/O Performance in Virtualization Environment,” *IEEE Trans. Cloud Comput.*, vol. 2, no. 2, pp. 130–142, 2014. [Online]. Available: <https://doi.org/10.1109/TCC.2014.2314649>
 - [24] F. Pace, D. Venzano, D. Carra, and P. Michiardi, “A Flexible Heuristic to Schedule Distributed Analytic Applications in Compute Clusters,” *IEEE Trans. Cloud Comput.*, vol. 11, no. 3, pp. 2217–2230, 2023. [Online]. Available: <https://doi.org/10.1109/TCC.2019.2926977>
 - [25] X. Tan and I. Hakala, “StateOS: A Memory-Efficient Hybrid Operating System for IoT Devices,” *IEEE Internet Things J.*, vol. 10, no. 11, pp. 9523–9533, 2023. [Online]. Available: <https://doi.org/10.1109/IIOT.2023.3234106>
 - [26] R. Rico, J. Rico-Azagra, and M. Gil-Martínez, “Hardware and RTOS Design of a Flight Controller for Professional Applications,” *IEEE Access*, vol. 10, pp. 134 870–134 883, 2022. [Online]. Available: <https://doi.org/10.1109/ACCESS.2022.3232749>
 - [27] C. Zhang, Y. Deng, H. Zhao, T. Chen, and S. Deng, “Tail-learning: Adaptive learning method for mitigating tail latency in autonomous edge systems,” *ACM Transactions on Autonomous and Adaptive Systems*, vol. 20, no. 4, 2025. [Online]. Available: <https://doi.org/10.1145/3737289>
 - [28] Y. Sharma and S. Moulik, “HEAT: A heterogeneous multicore real-time scheduler with efficient energy and temperature management,” *ACM SIGAPP Applied Computing Review*, vol. 22, no. 2, pp. 34–43, 2022. [Online]. Available: <https://doi.org/10.1145/3558053.3558056>
 - [29] S. Moulik and Y. Sharma, “FRESH: Fault-tolerant real-time scheduler for heterogeneous multiprocessor platforms,” *Future Generation Computer Systems*, vol. 161, pp. 214–225, 2024. [Online]. Available: <https://doi.org/10.1016/j.future.2024.07.008>
 - [30] M. Reuven and Y. Wiseman, “Medium-Term Scheduler as a Solution for the Thrashing Effect,” *Comput. J.*, vol. 49, no. 3, pp. 297–309, 2006. [Online]. Available: <https://doi.org/10.1093/comjnl/bxl001>
 - [31] A. Kvalnes, D. Johansen, R. van Renesse, F. B. Schneider, and S. V. Valvag, “Omni-Kernel: An Operating System Architecture for Pervasive Monitoring and Scheduling,” *IEEE Trans. Parallel Distrib. Syst.*, vol. 26, no. 10, pp. 2849–2862, 2015. [Online]. Available: <https://doi.org/10.1109/TPDS.2014.2362540>
 - [32] D. Senapati, A. Sarkar, and C. Karfa, “Energy-Aware Real-Time Scheduling of Multiple Periodic DAGs on Heterogeneous Systems,” *IEEE Trans. Comput.-Aided Design Integr. Circuits Syst.*, vol. 42, no. 8, pp. 2447–2460, 2023. [Online]. Available: <https://doi.org/10.1109/TCAD.2022.3228504>
 - [33] C. Steiger, H. Walder, and M. Platzner, “Operating Systems for Reconfigurable Embedded Platforms: Online Scheduling of Real-Time Tasks,” *IEEE Trans. Comput.*, vol. 53, no. 11, pp. 1393–1407, 2004. [Online]. Available: <https://doi.org/10.1109/TC.2004.99>
 - [34] D. Derafshi, A. Norollah, M. Khosroanjani, and H. Beitollahi, “HRHS: A High-Performance Real-Time Hardware Scheduler,” *IEEE Trans. Parallel Distrib. Syst.*, vol. 31, no. 4, pp. 897–908, 2020. [Online]. Available: <https://doi.org/10.1109/TPDS.2019.2952136>
 - [35] V. G. Gaitan, N. C. Gaitan, and I. Ungurean, “CPU Architecture Based on a Hardware Scheduler and Independent Pipeline Registers,” *IEEE Trans. Very Large Scale Integr. (VLSI) Syst.*, vol. 23, no. 9, pp. 1661–1674, 2015. [Online]. Available: <https://doi.org/10.1109/TVLSI.2014.2346542>



Geunsik Lim received the B.S. degree in Computer Science and Engineering from Ajou University, South Korea, in 2003, and the M.S. and Ph.D. degrees in Electrical and Computer Engineering from Sungkyunkwan University, South Korea, in 2014 and 2023, respectively. He is currently a principal software engineer at Samsung Electronics in South Korea. His research interests span operating systems and Linux-kernel scheduling, system and performance optimization, low-latency and energy-efficient resource management for heterogeneous mobile and embedded platforms, software platform engineering, and on-device artificial intelligence, with an emphasis on techniques that are practical to deploy in production systems and that translate readily from large-scale servers to power-constrained edge hardware.